K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Bowrampet, Hyderabad, Telangana – 500043

PROJECT REPORT

SIMILARITY-BASED DUPLICATE RECORD DETECTION SYSTEM

A Data Science Project Report submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology

BACHELOR OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT TEAM	ROLL NUMBER
Allipuram Akhil	2420030540
D. Surya	2420030440
P. Ruthvik Reddy	2420030546

Under the esteemed guidance of
Ms. Bonagani Prathusha
Assistant Professor
Department of Computer Science and Engineering

Academic Year: 2026–2027
Course: Data Science (24AD3107)

DECLARATION

We hereby declare that the project report entitled “Similarity-Based Duplicate Record Detection System” is a record of the work carried out by us in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering at K L (Deemed to be) University.

The work presented in this report describes the implementation, experiments, and results of the project undertaken by the team. The report has been prepared with due acknowledgement of the technical literature and resources referred to during development.

We further declare that the experimental results included in this report are based on the supplied project execution artifacts and that the limitations of the generated dataset and evaluation are stated wherever relevant.

Name	Roll Number	Signature
Allipuram Akhil	2420030540	
D. Surya	2420030440	
P. Ruthvik Reddy	2420030546	

CERTIFICATE

This is to certify that the project report entitled “Similarity-Based Duplicate Record Detection System” is a bonafide work carried out by Allipuram Akhil (2420030540), D. Surya (2420030440), and P. Ruthvik Reddy (2420030546), students of the Department of Computer Science and Engineering, K L (Deemed to be) University, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology during the academic year 2026–2027.

The work has been completed under my guidance and supervision.

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD
Ms. Bonagani Prathusha Assistant Professor		

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Ms. Bonagani Prathusha, Assistant Professor, Department of Computer Science and Engineering, for her guidance, technical suggestions, and encouragement throughout the development of this project.

We thank the Department of Computer Science and Engineering and K L (Deemed to be) University for providing the academic environment and facilities required to carry out the project. We also acknowledge the support of our faculty members, classmates, and families during the completion of this work.

Finally, we thank everyone who contributed directly or indirectly to the successful completion of the Similarity-Based Duplicate Record Detection System.

Allipuram Akhil (2420030540)
D. Surya (2420030440)
P. Ruthvik Reddy (2420030546)

ABSTRACT

Duplicate records are a common data-quality problem in structured databases. The same customer or entity may be entered more than once with differences in spelling, capitalization, punctuation, phone-number formatting, email values, or postal addresses. Exact-match rules are often insufficient to identify these near-duplicate records. This project presents a similarity-based duplicate record detection system that combines data preprocessing, candidate-pair generation, multi-field similarity computation, supervised machine-learning classification, duplicate clustering, and an interactive dashboard.

The documented experimental dataset contains 503 customer records, comprising 350 original records and 153 injected duplicates. The injected duplicates include 65 exact copies and 88 near-duplicate records. Preprocessing handles missing values, normalizes text fields, and standardizes phone numbers. Candidate pairs are generated using a blocking key based on sorted name-token initials and phone prefixes, with a neighboring-name safety mechanism. Similarity features include Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine, and exact phone/email match indicators.

The system compares Logistic Regression, Random Forest, and XGBoost classifiers. The supplied execution reports 605 candidate pairs, 172 flagged duplicate pairs, and 357 representative records after consolidation. The dashboard reports 0.995 accuracy, 1.000 precision, 0.981 recall, and 0.990 F1-score. The displayed Logistic Regression confusion matrix contains 130 true negatives, zero false positives, one false negative, and 51 true positives. The dashboard extension is implemented using Streamlit and provides dataset selection or CSV upload, result visualization, pair inspection, and downloadable outputs.

The results demonstrate the feasibility of combining approximate string matching and machine learning in a practical data-cleaning workflow. However, the experiment uses generated data, and independent validation on real-world datasets is required before operational deployment.

Keywords: Duplicate detection, entity resolution, record linkage, string similarity, Levenshtein distance, Jaro–Winkler, TF–IDF, Logistic Regression, Random Forest, XGBoost.

INDEX

Chapter	Title	Contents
1	Introduction	Background, problem, scope, objectives, importance, target users
2	System Requirements	Hardware, software and development requirements
3	Technology Stack	Python, ML libraries, dashboard, data and version control
4	System Architecture	Architecture layers, workflow and deployment view
5	Design	Input schema, data flow, feature design and interface
6	Implementation	Preprocessing, blocking, similarity, classification, clustering
7	Features	Dashboard modules and user workflow
8	Testing	Functional, model and integration checks
9	Deployment	Environment setup and dashboard launch
10	Challenges and Limitations	Development issues, constraints and mitigations
11	Future Enhancements	Planned research and product improvements
12	Conclusion	Summary, achievements and skills learned
13	References	Books, papers and technical documentation
14	Appendices	Screenshots, sample schema, formulas and user guide

Note: Page numbering is generated by the word processor and may change when the report is edited.

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

Organizations maintain customer and entity records through websites, mobile applications, registration forms, service desks, point-of-sale systems, and legacy databases. During repeated data entry or data migration, the same real-world entity can be represented by multiple rows. These rows may differ only in capitalization, punctuation, spacing, abbreviations, spelling, or the format of contact information.

Duplicate records affect data quality. They can fragment customer histories, distort customer counts, cause repeated communications, increase storage and maintenance effort, and reduce the reliability of analytics. In many applications, a simple equality check is not sufficient because two records can refer to the same entity even when their values are not identical.

Record linkage, also called entity resolution or duplicate detection, attempts to determine whether two records describe the same real-world entity. This project uses multiple similarity measures to compare names, emails, phone numbers, and addresses. A machine-learning classifier combines the comparison evidence and predicts whether a candidate pair is a duplicate.

1.2 Problem Statement

The problem addressed in this project is the identification of exact and near-duplicate records in structured customer datasets when one or more attributes contain spelling errors, inconsistent formatting, missing values, or small textual variations. The system must reduce unnecessary comparisons, evaluate multiple fields, classify candidate pairs, and produce a cleaned dataset while retaining enough information for users to inspect the decisions.

1.3 Scope of the Project

- Structured customer records containing record_id, name, email, phone, and address fields.
- Generated customer data used for controlled evaluation, including exact and near-duplicate examples.
- Preprocessing and normalization of text and phone-number values.
- Candidate-pair generation through blocking and neighboring-name candidates.
- Similarity computation using character, token, vector, and exact-match features.
- Comparison of Logistic Regression, Random Forest, and XGBoost.
- Duplicate-pair reporting, record clustering, representative-record selection, and CSV export.
- An interactive Streamlit dashboard for execution, visualization, inspection, and download.

1.4 Objectives

1. Prepare customer records by validating required columns and normalizing comparison fields.
2. Generate a manageable set of candidate pairs using blocking and a neighboring-name safety mechanism.
3. Calculate multiple similarity features for names, email addresses, phone numbers, and postal addresses.
4. Compare three supervised classifiers using a common feature representation.
5. Evaluate the models using accuracy, precision, recall, F1-score, and confusion matrices.
6. Convert predicted duplicate links into record clusters and retain a representative record from each cluster.

7. Provide a dashboard that supports dataset upload or generation, pair inspection, visual analytics, and downloadable results.

1.5 Importance of the Application

The application provides a repeatable workflow for improving the quality of structured records. By combining several similarity signals, it can identify variations that exact matching may overlook. Candidate generation reduces the number of pairs requiring detailed comparison. The dashboard makes the output inspectable and provides a cleaned dataset that can be used for subsequent analysis.

1.6 Target Users / Audience

- Data analysts responsible for preparing customer datasets.
- Database administrators reviewing data-quality issues.
- Organizations consolidating customer or entity records from multiple sources.
- Students and researchers studying record linkage, similarity measures, and classification.

CHAPTER 2
SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The project is designed for local execution on a general-purpose development computer. The following configuration is a practical recommendation for running Python data-processing tasks, model evaluation, and the dashboard. It is not a benchmark of minimum hardware measured during the experiment.

Resource	Recommended configuration	Purpose
Processor	Modern Intel Core i5 / AMD Ryzen 5 or better	Data preprocessing and model execution
Memory	8 GB RAM or higher; 16 GB preferred	Holding dataframes, candidate pairs and model objects
Storage	At least 5 GB free SSD space	Source code, environment, datasets and exported files
Display	1366 × 768 or higher	Dashboard and result inspection
Network	Internet access for package installation	Installing dependencies; local dashboard runs on localhost

2.2 Software Requirements

Software	Role
Windows 10/11, Linux or macOS	Operating system for development
Python 3.x	Data processing and machine-learning runtime
pip / virtual environment	Dependency installation and isolation
Web browser	Accessing the local Streamlit dashboard
Git and GitHub (if used)	Source-code version control and collaboration

2.3 Development Tools and Frameworks

- Streamlit: interactive Python dashboard and user controls.
- Pandas and NumPy: tabular data processing and numerical operations.
- scikit-learn: feature utilities, model training, splitting, and evaluation.
- XGBoost: gradient-boosted tree classifier.
- Python-Levenshtein and jellyfish: string similarity calculations.
- Plotly: interactive charts in the dashboard.
- Matplotlib and Seaborn: supporting data visualization.

CHAPTER 3
TECHNOLOGY STACK

3.1 Programming Language

Python is used for the data-processing pipeline, similarity computation, machine-learning models, and dashboard integration. Its data-science ecosystem supports the project's structured-data workflow.

3.2 Data Processing Libraries

Pandas is used to load CSV files, validate columns, normalize fields, construct candidate-pair tables, and prepare downloadable outputs. NumPy supports numerical operations used by the processing and modelling stages.

3.3 Similarity and Machine-Learning Libraries

scikit-learn supports TF–IDF vectorization, classification utilities, data splitting, and evaluation metrics. Python-Levenshtein and jellyfish support string-comparison functions. XGBoost provides the gradient-boosted classifier.

3.4 Dashboard and Visualization

The supplied dashboard add-on uses Streamlit to create the interactive web interface. Plotly is used to display similarity-score distributions and dataset-reduction charts. Streamlit controls allow the user to select a dataset source, upload a CSV file, adjust a threshold for unlabelled data, start processing, inspect results, and download generated CSV files.

3.5 Data Storage and Version Control

The project uses CSV files as the input and output data format. The dashboard expects the required columns record_id, name, email, phone, and address. If a duplicate_of ground-truth column is present, it can be used to construct labels for supervised evaluation. Git/GitHub may be used to manage source-code changes; repository details should be inserted by the project team if required by the department.

3.6 Technology Summary

Layer	Technology	Responsibility
Language	Python	Pipeline and model logic
Data	Pandas, NumPy	Loading, cleaning and feature tables
Similarity	Levenshtein, jellyfish	Character-level comparison
Machine learning	scikit-learn, XGBoost	Pair classification and evaluation
Dashboard	Streamlit	Interactive user interface
Visualization	Plotly, Matplotlib, Seaborn	Charts and analytical views
Input/output	CSV	Dataset, candidate pairs, reports and cleaned records

CHAPTER 4
SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The system follows a staged processing architecture. Each stage prepares information for the next stage, beginning with raw records and ending with predicted duplicate pairs, consolidated records, and dashboard outputs.

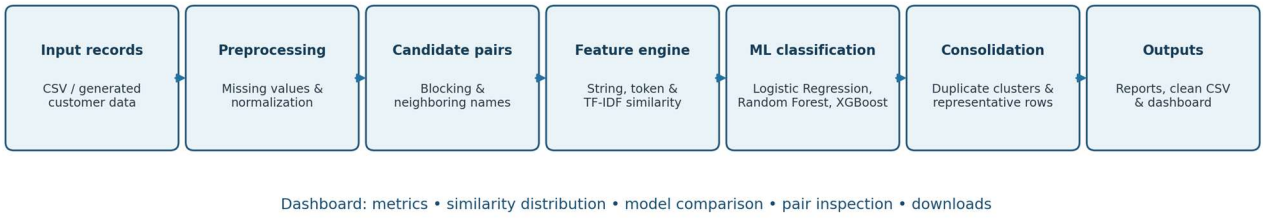


Figure 4.1: Functional architecture of the Similarity-Based Duplicate Record Detection System.

4.2 Description of Each Layer

1. Data ingestion layer

Reads the project dataset or a user-uploaded CSV file. It validates the required record fields and checks that record identifiers are unique and numeric.

2. Preprocessing layer

Handles missing text values and creates normalized comparison values. Text fields are converted to a consistent case, whitespace is normalized, and phone values are standardized by retaining digits.

3. Candidate-generation layer

Creates candidate record pairs using blocking based on sorted name-token initials and phone prefixes. A neighboring-name mechanism supplements the blocking process.

4. Similarity feature layer

Computes character-level, token-level, vector-space, and exact-match signals for each candidate pair.

5. Classification layer

Evaluates Logistic Regression, Random Forest, and XGBoost using the pair-level feature representation when usable ground-truth labels are available.

6. Consolidation layer

Connects predicted duplicate pairs into clusters and selects a representative record with fewer missing fields.

7. Presentation and output layer

Displays summary metrics, score distributions, model evaluation, confusion-matrix information, detected pairs, and downloadable CSV outputs.

4.3 Pair-Level Matching and Record-Level Consolidation

Pair-level matching determines whether two candidate records are likely to describe the same entity. Record-level consolidation combines the predicted links into connected components. One record can participate in multiple duplicate pairs, so the number of flagged pairs is not necessarily equal to the number of rows removed from the dataset.

4.4 Deployment Architecture

The supplied dashboard extension is intended for local execution. The user launches the Streamlit application from the project environment and accesses the local URL printed by Streamlit, normally http://localhost:8501. A production deployment would require additional authentication, access control, audit logging, secure transport, and safeguards for reviewing and reversing record merges.

CHAPTER 5
DESIGN

5.1 Input Data Design

The dashboard requires a CSV file with the following fields. The record_id value must be unique and numeric. The duplicate_of field is optional and is used only when available as ground-truth information for supervised evaluation.

Field	Description	Requirement
record_id	Unique identifier for each record	Required
name	Customer or entity name	Required
email	Email address	Required
phone	Phone number	Required
address	Postal or contact address	Required
duplicate_of	Reference identifier for the source record of an injected duplicate	Optional

5.2 Data Flow

8. The user selects the bundled dataset or uploads a CSV file.
9. The application validates the input structure and record identifiers.
10. The preprocessing module normalizes the record fields.
11. The blocking stage creates candidate pairs for detailed comparison.
12. The feature engine computes similarity values for each pair.
13. The classification or threshold decision marks candidate pairs as duplicates or non-duplicates.
14. The consolidation module groups linked records and selects representative rows.
15. The dashboard displays results and enables CSV downloads.

5.3 Database / Data Schema

The supplied implementation uses CSV-based data exchange rather than a relational database schema. The key analytical artifacts are the raw record dataset, the candidate-pair score table, the duplicate-pair report, and the cleaned unique-record dataset.

Artifact	Purpose
raw_records.csv	Input records used by the pipeline
candidate_pair_scores.csv	Candidate pairs, similarity features, labels and predictions
duplicate_records_report.csv	Pairs classified as duplicates, with record details and similarity evidence
cleaned_unique_dataset.csv	Representative records retained after clustering
model_performance_metrics.json	Model evaluation metrics when ground-truth labels support training

5.4 User Interface Design

The dashboard is organized around an execution workflow. The sidebar contains dataset-source controls, a CSV uploader, an adjustable similarity threshold for unlabelled data, and a run button. The main view presents summary indicators, charts, model evaluation, a detected-pairs table, and download controls.

CHAPTER 6
IMPLEMENTATION

6.1 Module-Wise Implementation

Module	Main responsibility
Dataset loading and validation	Read the default or uploaded CSV and validate required columns.
Preprocessing	Handle missing fields and normalize names, emails, addresses and phone numbers.
Candidate generation	Create likely record pairs using blocking and neighboring-name candidates.
Feature engineering	Build pair-level similarity features and a combined score.
Model training and evaluation	Split labelled pairs, train classifiers, and calculate evaluation metrics.
Duplicate report	Join flagged pair IDs with source record values for inspection.
Clustering and cleaning	Use connected components and select a representative row for each cluster.
Dashboard and exports	Display metrics and charts, support search, and download CSV files.

6.2 Data Preprocessing

The application first checks that the required columns are present. It converts record_id values to numeric integers and rejects duplicate identifiers. Missing values in name, email, phone, and address are converted to empty strings for the comparison workflow. The preprocessing module then creates normalized representations used by the candidate-generation and similarity stages.

Names, emails, and addresses are normalized to reduce case and spacing differences. Phone numbers are standardized by retaining digits only. Normalization reduces false differences caused by formatting, but it does not establish that two records belong to the same person.

6.3 Candidate-Pair Generation

Exhaustively comparing every unordered pair in a dataset of 503 records would require $503 \times 502 / 2 = 126{,}253$ comparisons. The implemented candidate-generation strategy reduces this work by grouping records that share a blocking key. The documented key uses sorted first letters of name tokens together with the first three digits of the normalized phone number. A neighboring-name safety mechanism adds adjacent records after sorting by normalized name.

The supplied execution artifact contains 605 candidate pairs. Blocking improves efficiency, but it can exclude true duplicate pairs if the relevant records do not share a candidate-generation key. Candidate recall should therefore be measured in future evaluation.

6.4 Similarity Feature Engineering

Each candidate pair is represented by multiple comparison features. Character-level measures capture spelling differences, token-level measures capture word overlap, vector-space similarity compares textual content, and exact-match indicators capture equality of normalized contact fields.

Feature	Purpose

name_levenshtein	Character-edit similarity between names
name_jaro_winkler	Short-name similarity with emphasis on common prefixes
email_levenshtein	Character-edit similarity between email values
address_levenshtein	Character-edit similarity between addresses
address_jaro_winkler	Address string similarity
name_jaccard	Token overlap between names
address_jaccard	Token overlap between addresses
phone_exact_match	Equality of normalized phone values
email_exact_match	Equality of normalized email values
tfidf_cosine	Cosine similarity between TF–IDF text vectors
combined_score	Combined pair-level similarity signal

6.5 Machine-Learning Classification

The application compares Logistic Regression, Random Forest, and XGBoost using the same pair-level feature representation. When the optional duplicate_of field provides enough examples from both classes, labels are constructed and the models are trained and evaluated. The project design documents a 70/30 train-test split and grid search with five-fold cross-validation. The final report does not claim exact hyperparameter values because they are not established by the dashboard screenshots.

6.6 Duplicate Clustering and Representative Selection

Predicted duplicate pairs are treated as edges in a graph. A union-find structure joins the sets containing the two records in each predicted pair. The resulting connected components represent record clusters. The implementation counts missing values across the name, email, phone, and address fields and retains a record with fewer missing fields as the representative of its cluster.

A false-positive pair can connect two distinct entities and propagate through a cluster. For this reason, the dashboard documentation advises users to review flagged pairs before deleting or merging records.

6.7 Dashboard Integration

The Streamlit dashboard calls the existing preprocessing, candidate-generation, feature-engineering, and model modules. It presents four summary metrics, similarity-score distribution, dataset reduction, model evaluation, confusion-matrix details, a searchable detected-pairs table, and download buttons for duplicate pairs, cleaned records, and candidate scores.

CHAPTER 7
FEATURES

7.1 Dataset Selection and Upload

Users can choose the project dataset or upload a CSV file. Uploaded data must contain record_id, name, email, phone, and address. If a duplicate_of field is present and contains enough examples from both classes, it can support supervised evaluation.

7.2 Data Validation and Preprocessing

The dashboard validates the required columns, checks that record identifiers are unique, and converts missing text values into empty strings before invoking the preprocessing pipeline.

7.3 Candidate-Pair and Similarity Analysis

The candidate-generation and feature-engineering modules prepare pairs and calculate multiple similarity signals. The dashboard visualizes the combined-score distribution so users can inspect how flagged and non-flagged pairs are distributed.

7.4 Model Evaluation

When usable labels are available, the dashboard displays the model evaluation table, comparative metric chart, and confusion matrix for the selected best model. For uploaded data without ground-truth labels, the application uses the adjustable combined-similarity threshold and does not claim accuracy, precision, or recall.

7.5 Duplicate-Pair Inspection

The detected-pairs section provides a searchable table containing the two record identifiers, their field values, and similarity evidence. The search field can be used to locate values across names, emails, phone numbers, and addresses.

7.6 Dataset Cleaning and Downloads

The application provides downloads for the duplicate-pair report, cleaned unique-record dataset, and candidate-pair score table. The cleaned dataset retains one representative record for each connected duplicate cluster.

7.7 User Workflow

16. Open the dashboard in a browser after launching the Streamlit application.
17. Choose the bundled dataset or upload a CSV file.
18. For unlabelled data, select an appropriate similarity threshold.
19. Start duplicate detection and wait for processing to complete.
20. Review the summary metrics, score distribution, model evaluation, and flagged pairs.
21. Inspect uncertain pairs and validate them before using the cleaned dataset.
22. Download the duplicate report, cleaned dataset, or candidate-pair scores.

CHAPTER 8
TESTING

8.1 Testing Approach

Testing is considered at the data-validation, preprocessing, candidate-generation, feature-engineering, model-evaluation, clustering, and dashboard-output stages. The supplied evidence includes a completed dashboard run and generated output artifacts. The report distinguishes displayed evaluation metrics from independent software-test coverage.

8.2 Input Validation Checks

Test	Input / condition	Expected behavior
Required-column validation	One or more required columns missing	Reject input and identify missing columns
Record ID validation	Non-numeric record_id	Reject invalid identifier values
Unique identifier check	Repeated record_id values	Reject duplicate identifiers
Missing text fields	Null name/email/phone/address	Convert to empty strings for processing
No candidate pairs	Dataset produces no blocking candidates	Display a no-candidate warning
Unlabelled upload	No duplicate_of field	Use selected similarity threshold; do not report supervised metrics

8.3 Model Evaluation Metrics

Accuracy measures the fraction of correctly classified candidate pairs. Precision measures the fraction of predicted duplicate pairs that are correct. Recall measures the fraction of actual duplicate pairs that are identified. F1-score is the harmonic mean of precision and recall.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-score = 2 × Precision × Recall / (Precision + Recall)

8.4 Observed Execution Results

Measure	Observed value
Input records	503
Candidate pairs	605
Flagged duplicate pairs	172
Unique records after cleaning	357
Records removed from output	146
Record-count reduction	29.0%

Flagged-pair rate	28.4%

The 146-row reduction is calculated as 503 − 357. The record-count reduction is approximately (146 / 503) × 100 = 29.0%. The flagged-pair rate is approximately (172 / 605) × 100 = 28.4%. These are different measures because one record can participate in multiple flagged pairs.

8.5 Classifier Results

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.995	1.000	0.981	0.990
Random Forest	0.995	1.000	0.981	0.990
XGBoost	0.995	1.000	0.981	0.990

The dashboard reports the same rounded values for the three models. Logistic Regression is identified by the application as the best evaluated model. Since the displayed metrics are rounded to three decimal places, the screenshots do not establish whether unrounded scores are identical.

8.6 Confusion Matrix

Actual / Predicted	Predicted non-duplicate	Predicted duplicate
Actual non-duplicate	130 (TN)	0 (FP)
Actual duplicate	1 (FN)	51 (TP)

The displayed confusion matrix covers 182 evaluated candidate pairs. These counts produce accuracy 0.9945, precision 1.0000, recall 0.9808, and F1-score approximately 0.9903, which round to the dashboard values.

8.7 Testing Limitations

The current report does not claim a 100% automated software-test pass rate because the available execution evidence does not establish such a test suite. Independent testing should include repeated runs, edge cases, malformed CSV inputs, blocking recall measurement, entity-level train-test separation, and human verification of flagged pairs.

CHAPTER 9
DEPLOYMENT

9.1 Environment Setup

The dashboard add-on is installed into the project's existing Python virtual environment. From the project root, install the dashboard dependencies using the provided requirements-dashboard.txt file.

```
python -m pip install -r requirements-dashboard.txt
```

9.2 Launching the Dashboard

From the project root directory, run the Streamlit application:

```
python -m streamlit run dashboard/app.py
```

Open the local address printed in the terminal. The setup notes identify http://localhost:8501 as the normal local address.

9.3 CSV Upload Format

CSV uploads must include record_id, name, email, phone, and address. The record_id column must contain unique numeric identifiers. The optional duplicate_of column can support supervised evaluation when enough examples of both classes are available.

9.4 Deployment Considerations

The supplied extension documents local dashboard execution. Public hosting, cloud deployment, authentication, encrypted transport, access control, audit trails, and production database integration are not established as completed features in the supplied project artifacts. These should be implemented and tested before production use.

CHAPTER 10
CHALLENGES AND LIMITATIONS

10.1 Data Quality Challenges

Customer attributes can contain missing values, inconsistent capitalization, spacing differences, punctuation, spelling errors, and alternate phone-number formats. These variations make exact matching unreliable and require a consistent preprocessing strategy.

10.2 Candidate Generation Challenges

Blocking reduces the number of pairs that require detailed comparison. However, strict blocking can exclude true duplicates when the values used to create the key are corrupted. The neighboring-name mechanism is intended to recover some likely pairs that fall across a blocking boundary.

10.3 Model and Evaluation Challenges

The displayed model metrics are based on a generated dataset and a limited evaluated sample. The same rounded metrics appear for all three models. The screenshots do not establish unrounded model differences, the full random seed, or the exact split construction. Pair-level splitting may also cause leakage if related records appear across both partitions.

10.4 Clustering Risks

A false-positive pair can merge otherwise distinct entities through connected-component clustering. Selecting a representative based on missing-field count does not resolve conflicting values across records. Human review and reversible merge operations are recommended before any destructive cleaning.

10.5 Current Limitations

- The benchmark is generated and may not represent the full diversity of real customer data.
- The exact normalization and weighting formula for combined_score is implementation-specific.
- Candidate pairs excluded by blocking cannot be recovered by the classifier.
- The available screenshots do not report blocking recall.
- The cleaned output selects a representative row rather than reconciling conflicting field values.
- The current dashboard is an experimental local tool and is not evidence of production security or scalability.

CHAPTER 11
FUTURE ENHANCEMENTS

11.1 Larger and More Diverse Datasets

Evaluate the system on larger independently labelled datasets and datasets containing multilingual names, shared phone numbers, reused email addresses, incomplete addresses, and realistic data-entry errors.

11.2 Improved Candidate Generation

Compare multiple blocking keys, sorted-neighborhood indexing, phonetic encodings, and adaptive blocking. Measure candidate reduction ratio and candidate recall to quantify the efficiency-quality trade-off.

11.3 Model and Threshold Improvements

Evaluate calibrated probabilities, precision-recall curves, threshold sensitivity, entity-level train-test splitting, repeated cross-validation, and confidence intervals. Explore transformer-based entity matching as an extension while retaining the current interpretable baseline.

11.4 Human Review and Auditability

Add a reviewer workflow for uncertain matches, field-level survivorship rules, reversible merge operations, and audit logs. The system should preserve original values and explain why a pair was flagged.

11.5 Incremental Processing and Deployment

Explore incremental matching for newly arriving records, database integration, user authentication, role-based access control, secure file handling, and scalable deployment. These are proposed enhancements and are not claimed as completed features.

CHAPTER 12
CONCLUSION

12.1 Project Summary

The Similarity-Based Duplicate Record Detection System demonstrates an end-to-end workflow for identifying exact and near-duplicate structured customer records. It combines data validation, preprocessing, blocking, multi-field similarity computation, supervised classification, duplicate clustering, and a dashboard for inspecting and exporting results.

12.2 What Was Achieved

- Processed a generated dataset containing 503 records.
- Prepared 605 candidate pairs for detailed comparison.
- Flagged 172 candidate pairs as duplicates in the supplied run.
- Compared Logistic Regression, Random Forest, and XGBoost.
- Reported 0.995 accuracy, 1.000 precision, 0.981 recall, and 0.990 F1-score on the displayed evaluation.
- Produced 357 representative records after consolidation, reducing the record count by approximately 29.0%.
- Provided a Streamlit dashboard for dataset input, result visualization, pair inspection, and downloads.

12.3 Skills Learned

The project provided practical experience in data preprocessing, approximate string matching, feature engineering, candidate generation, supervised classification, evaluation metrics, graph-based clustering, interactive dashboard development, and interpretation of data-quality results.

12.4 Final Remarks

The results are promising for the supplied generated dataset, but they should not be interpreted as proof of performance on all real-world customer databases. Independent validation, improved blocking evaluation, and human review are necessary steps before operational deployment.

CHAPTER 13
REFERENCES

[1] I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” Journal of the American Statistical Association, vol. 64, no. 328, pp. 1183–1210, 1969.

[2] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, pp. 707–710, 1966.

[3] M. A. Jaro, “Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida,” Journal of the American Statistical Association, vol. 84, no. 406, pp. 414–420, 1989.

[4] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in Proc. Section on Survey Research Methods, American Statistical Association, 1990, pp. 354–359.

[5] G. Salton and M. J. McGill, Introduction to Modern Information Retrieval. New York, NY, USA: McGraw-Hill, 1983.

[6] P. Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Berlin, Germany: Springer, 2012.

[7] P. Christen, “A survey of indexing techniques for scalable record linkage and deduplication,” IEEE Transactions on Knowledge and Data Engineering, vol. 24, no. 9, pp. 1537–1555, 2012.

[8] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, “Blocking and filtering techniques for entity resolution: A survey,” ACM Computing Surveys, vol. 53, no. 2, 2020.

[9] M. Bilenko and R. J. Mooney, “Adaptive duplicate detection using learnable string similarity measures,” in Proc. ACM SIGKDD, 2003, pp. 39–48.

[10] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

[11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016, pp. 785–794.

[12] S. Mudgal et al., “Deep learning for entity matching: A design space exploration,” in Proc. ACM SIGMOD, 2018, pp. 19–34.

[13] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, “Deep entity matching with pre-trained language models,” arXiv:2004.00584, 2020.

[14] scikit-learn Developers, “scikit-learn: Machine learning in Python,” https://scikit-learn.org/.

[15] Streamlit, “Streamlit documentation,” https://docs.streamlit.io/.

[16] XGBoost Developers, “XGBoost documentation,” https://xgboost.readthedocs.io/.

CHAPTER 14
APPENDICES

Appendix A: Dashboard Screenshots

The following figures are taken from the supplied project execution screenshots. They illustrate the observed run and dashboard views.

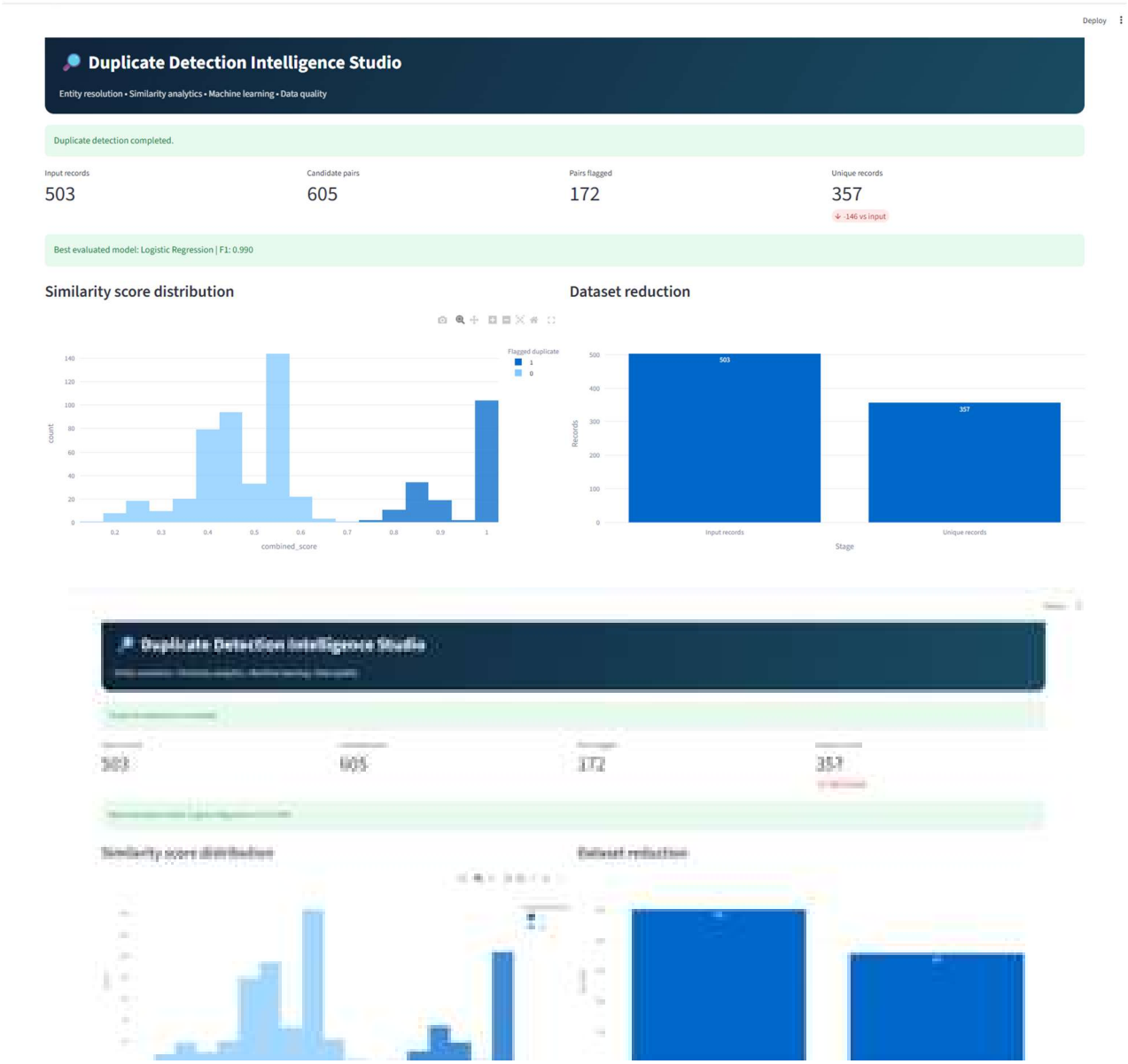
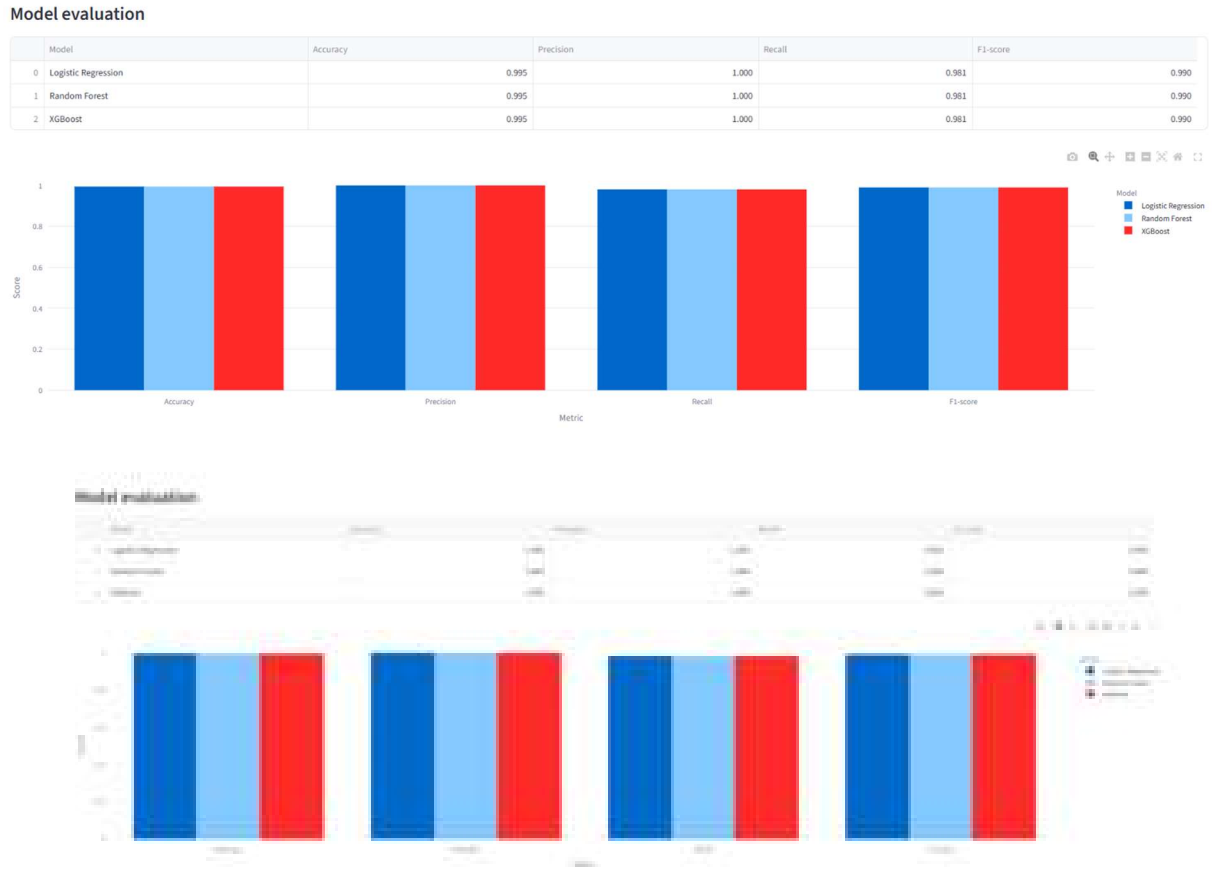


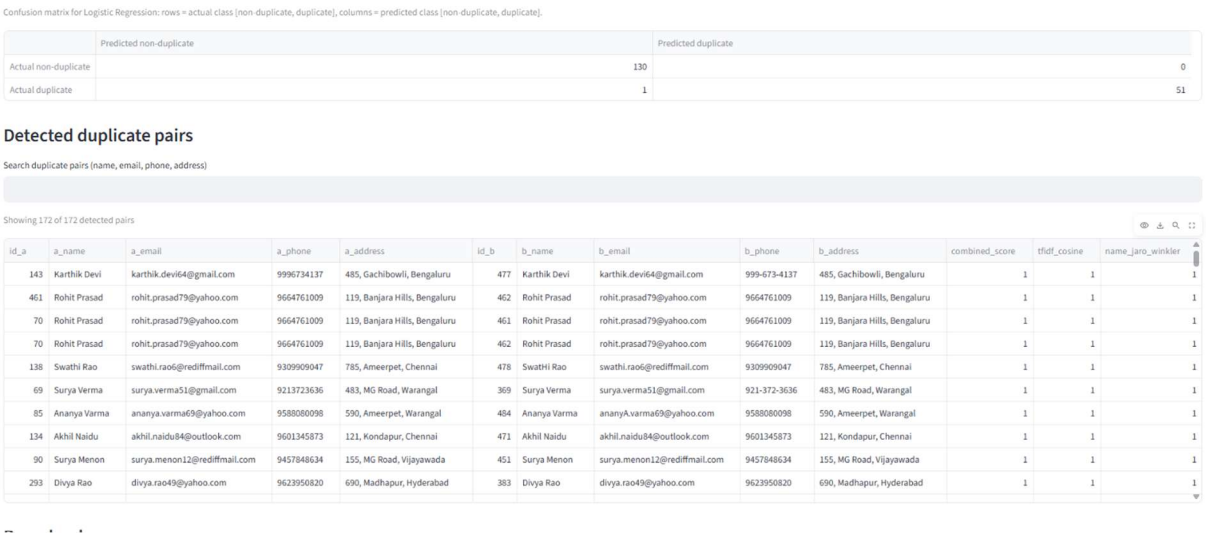
Figure A.1: Dashboard overview showing 503 input records, 605 candidate pairs, 172 flagged pairs, and 357 unique records.



Model evaluation

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.995	1.000	0.981	0.990
1	Random Forest	0.995	1.000	0.981	0.990
2	XGBoost	0.995	1.000	0.981	0.990

Figure A.2: Model evaluation table and comparative classifier metrics.



Confusion matrix for Logistic Regression: rows = actual class [non-duplicate, duplicate], columns = predicted class [non-duplicate, duplicate].

	Predicted non-duplicate	Predicted duplicate
Actual non-duplicate	130	0
Actual duplicate	1	51

Detected duplicate pairs

Search duplicate pairs (name, email, phone, address)

Showing 172 of 172 detected pairs

id_a	a_name	a_email	a_phone	a_address	id_b	b_name	b_email	b_phone	b_address	combined_score	tfidf_cosine	name_jaro_winkler
143	Karthik Devi	karthik.devi64@gmail.com	9996734137	485, Gachibowli, Bengaluru	477	Karthik Devi	karthik.devi64@gmail.com	999-673-4137	485, Gachibowli, Bengaluru	1	1	1
461	Rohit Prasad	rohit.prasad79@yahoo.com	9664761009	119, Banjara Hills, Bengaluru	462	Rohit Prasad	rohit.prasad79@yahoo.com	9664761009	119, Banjara Hills, Bengaluru	1	1	1
70	Rohit Prasad	rohit.prasad79@yahoo.com	9664761009	119, Banjara Hills, Bengaluru	461	Rohit Prasad	rohit.prasad79@yahoo.com	9664761009	119, Banjara Hills, Bengaluru	1	1	1
70	Rohit Prasad	rohit.prasad79@yahoo.com	9664761009	119, Banjara Hills, Bengaluru	462	Rohit Prasad	rohit.prasad79@yahoo.com	9664761009	119, Banjara Hills, Bengaluru	1	1	1
138	Swathi Rao	swathi.rao6@rediffmail.com	9309909047	785, Ameerpet, Chennai	478	SwatHi Rao	swathi.rao6@rediffmail.com	9309909047	785, Ameerpet, Chennai	1	1	1
69	Surya Verma	surya.verma51@gmail.com	9213723636	483, MG Road, Warangal	369	Surya Verma	surya.verma51@gmail.com	921-372-3636	483, MG Road, Warangal	1	1	1
85	Ananya Varma	ananya.varma69@yahoo.com	9588080098	590, Ameerpet, Warangal	484	Ananya Varma	ananyA.varma69@yahoo.com	9588080098	590, Ameerpet, Warangal	1	1	1
134	Akhil Naidu	akhil.naidu84@outlook.com	9601345873	121, Kondapur, Chennai	471	Akhil Naidu	akhil.naidu84@outlook.com	9601345873	121, Kondapur, Chennai	1	1	1
90	Surya Menon	surya.menon12@rediffmail.com	9457848634	155, MG Road, Vijayawada	451	Surya Menon	surya.menon12@rediffmail.com	9457848634	155, MG Road, Vijayawada	1	1	1
293	Divya Rao	divya.rao49@yahoo.com	9623950820	690, Madhapur, Hyderabad	383	Divya Rao	divya.rao49@yahoo.com	9623950820	690, Madhapur, Hyderabad	1	1	1

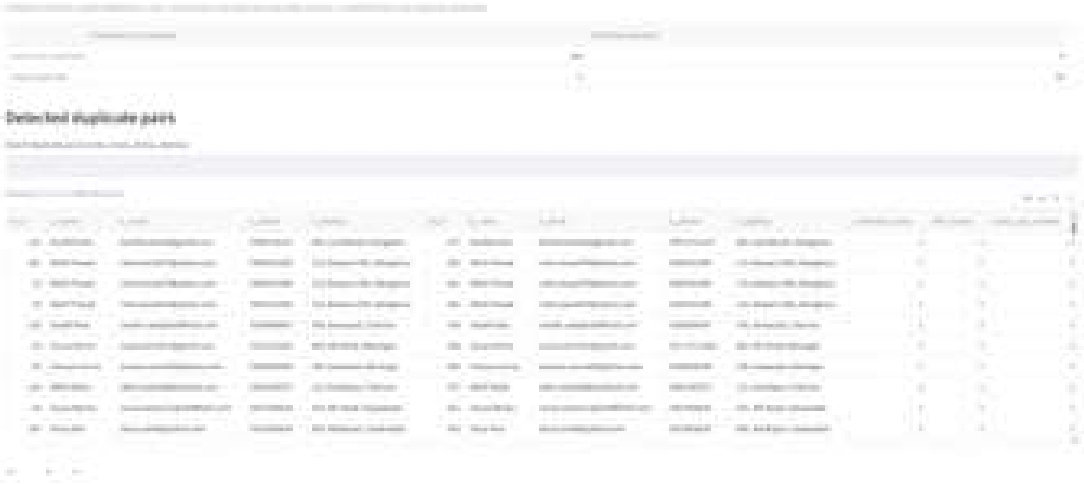


Figure A.3: Detected duplicate-pair table for record-level inspection.

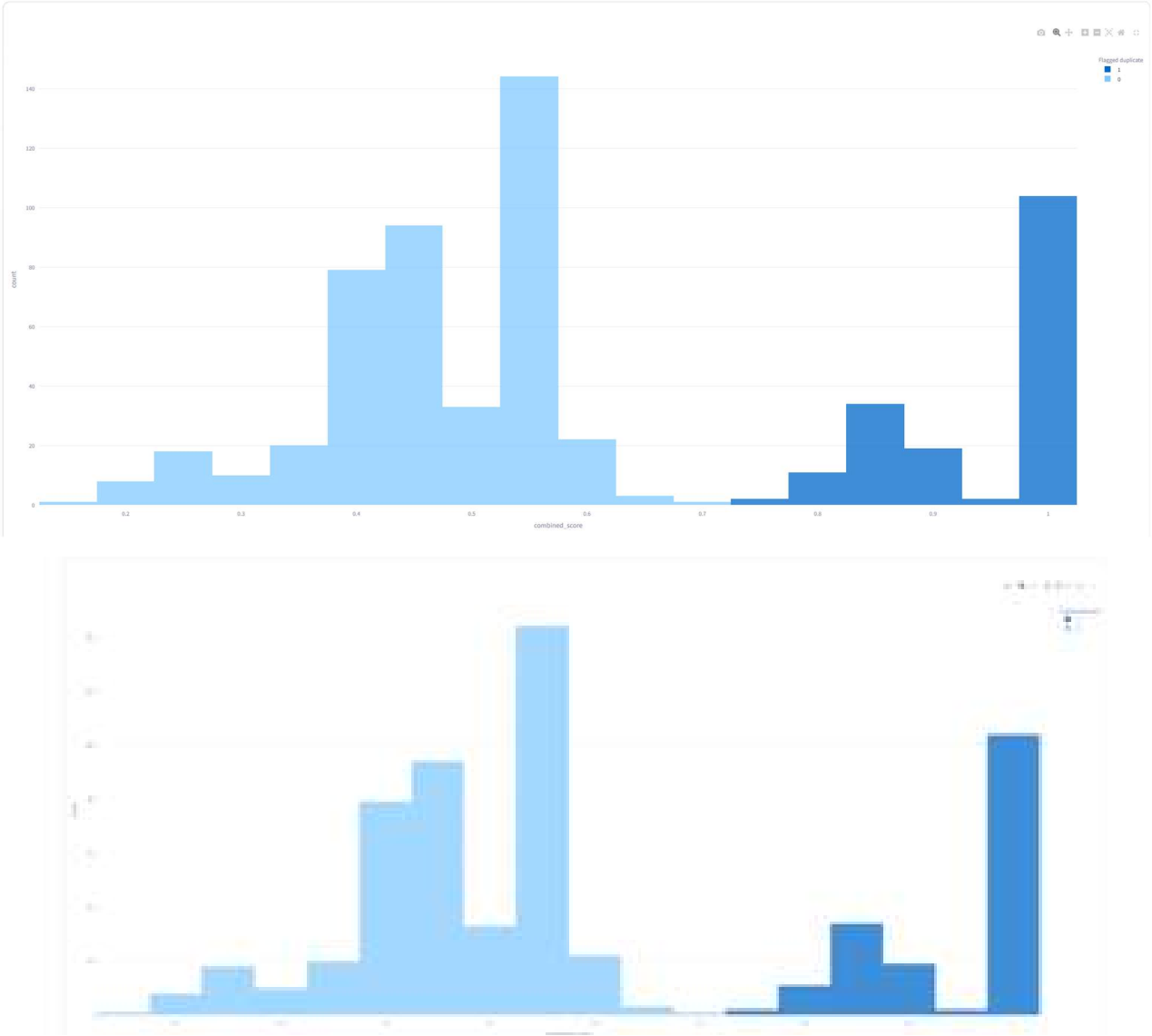


Figure A.4: Combined similarity-score distribution.

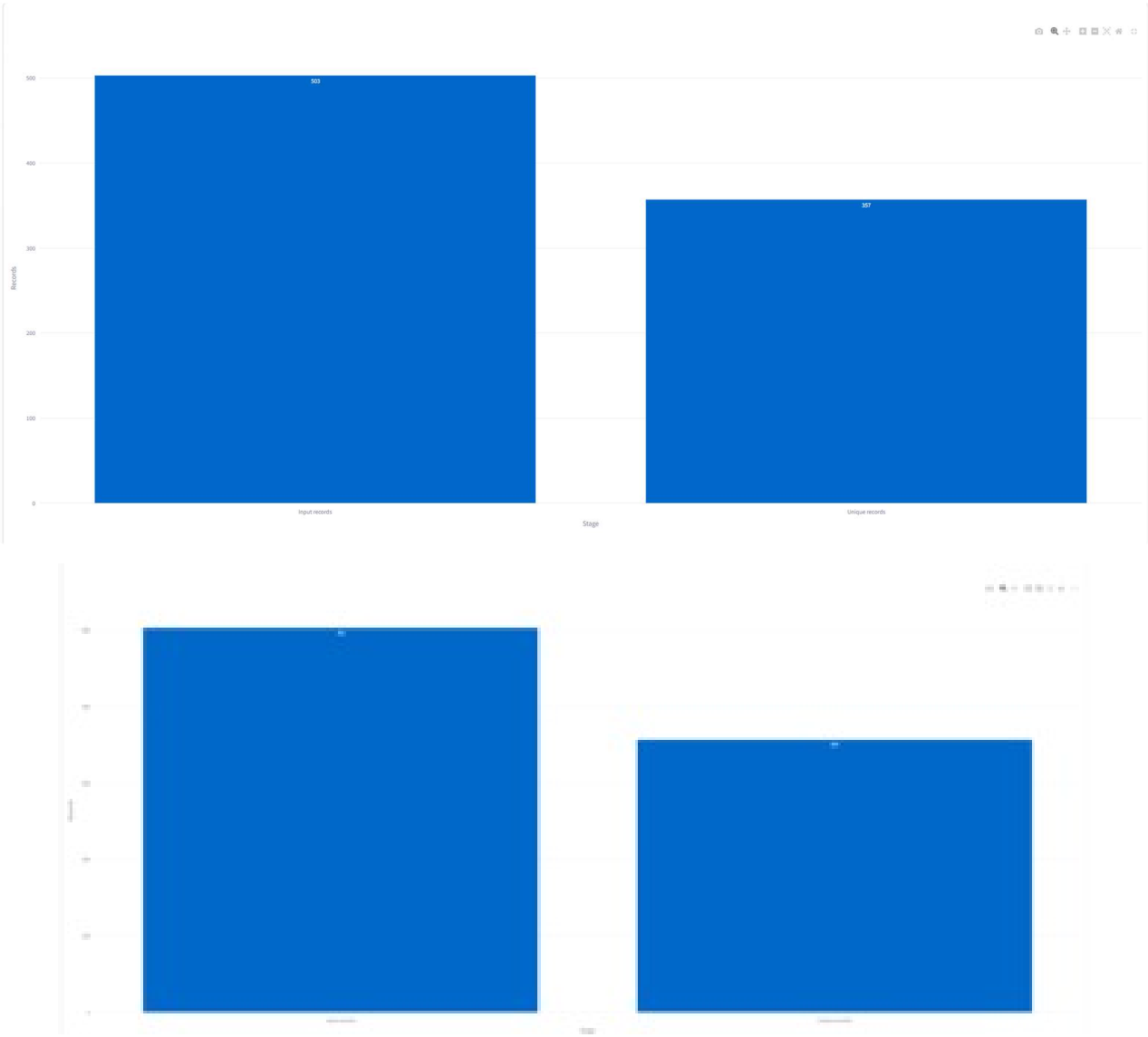
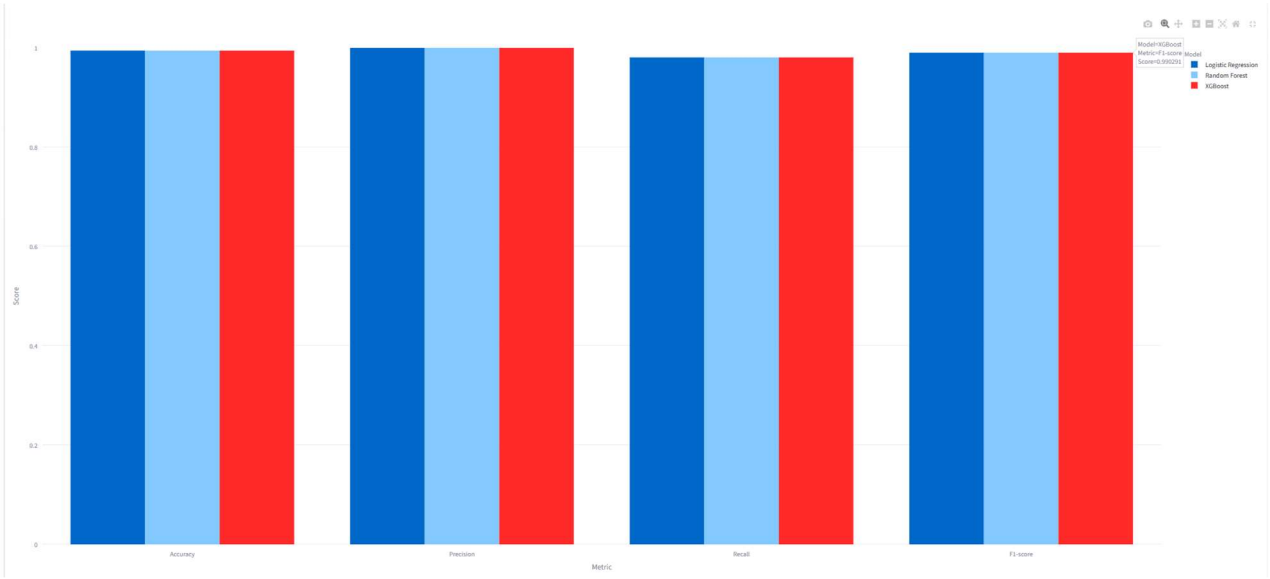


Figure A.5: Dataset reduction chart.



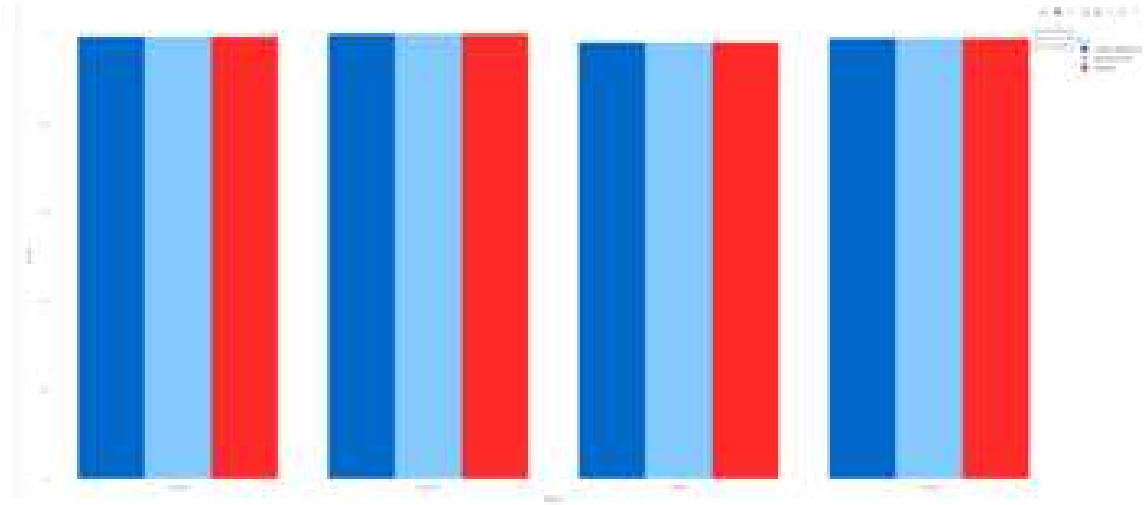


Figure A.6: Confusion-matrix / model evaluation visualization.

Figure A.7: Dataset preview / input records used by the dashboard.



Figure A.8: Additional dashboard view from the supplied execution screenshots.

Figure A.9: Functional pipeline architecture supplied with the project.

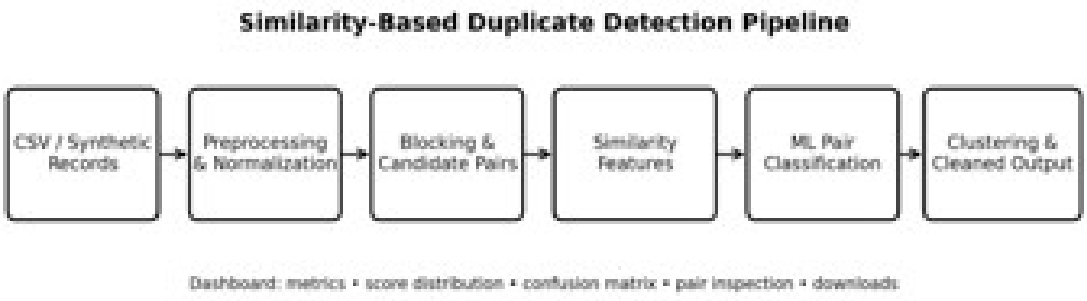
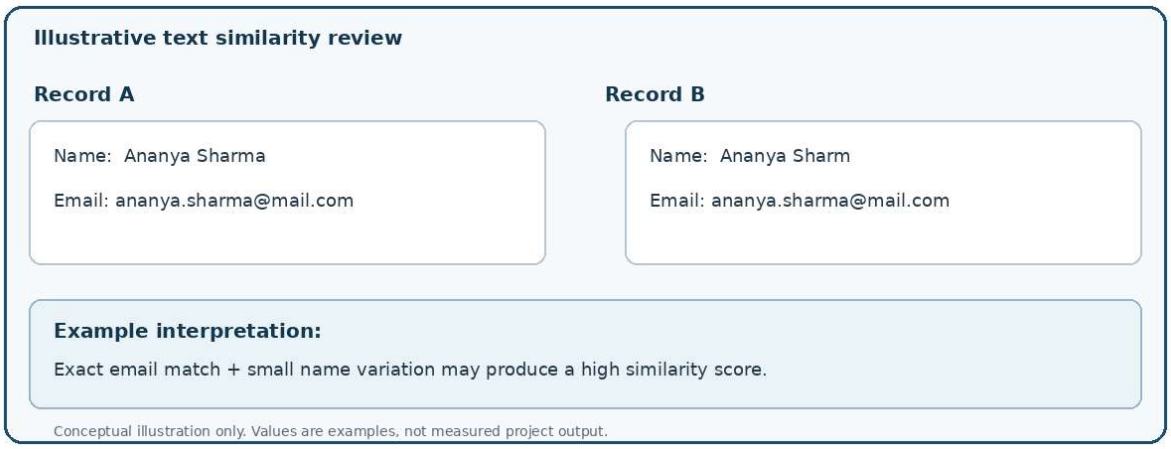


Figure A.10: Illustrative example of text-field similarity review (conceptual, not a captured application result).



Appendix B: Mathematical Formulations

Normalized Levenshtein similarity can be expressed as:

$$S_{lev}(a,b) = 1 - d(a,b) / \max(|a|, |b|),$$

where $d(a,b)$ is the Levenshtein edit distance. Empty-string cases require an explicit implementation rule.

Jaccard similarity between token sets A and B is:

$$J(A,B) = |A \cap B| / |A \cup B|.$$

Cosine similarity between TF–IDF vectors u and v is:

$$\cos(u,v) = (u \cdot v) / (||u|| \, ||v||).$$

The project also uses Jaro–Winkler similarity and exact-match indicators for normalized phone and email values. The combined-score weights are implementation-specific and are not reproduced here because the supplied report artifacts do not establish the exact weighting formula.

Appendix C: User Guide

23. Install the dashboard dependencies into the project virtual environment.
24. Run the Streamlit command from the project root.
25. Select the project dataset or upload a correctly structured CSV.
26. Set a similarity threshold only when the uploaded data has no usable ground-truth labels.
27. Run duplicate detection and inspect the summary metrics.
28. Review flagged pairs and verify uncertain cases before merging or deleting records.
29. Download the required CSV output files.

Appendix D: Sample CSV Header

```
record_id,name,email,phone,address,duplicate_of
```

The duplicate_of column is optional. When it is present, it is used as ground-truth relationship information for supervised evaluation and is not an ordinary prediction feature.

Appendix E: Academic and Experimental Disclosure

The numerical results in this report are transcribed from the supplied project abstract, IEEE-style research paper, dashboard extension, and execution screenshots. The dataset is generated for controlled experimentation. The dashboard metrics are reported as displayed and should not be interpreted as independently validated performance on a real production population. The project report intentionally does not claim unverified software versions, cloud deployment, authentication, or a formal automated-test pass rate.